

CS NETWORKING LAB - S6 B.TECH (BIT NOTE)

1. TCP Socket

Server

```
#include<stdio.h>
#include<string.h>
#include<sys/socket.h>
#include<arpa/inet.h>
#include<unistd.h>
int main(){
    int ssock, csock, csize;
    struct sockaddr_in saddr, caddr;
    char msg[2000]="Hello!";
    ssock = socket(AF_INET, SOCK_STREAM, 0);
    saddr.sin_family = AF_INET;
    saddr.sin_port = htons(2000);
    saddr.sin_addr.s_addr =
inet_addr("127.0.0.1");
    bind(ssock, (struct sockaddr*)&saddr,
sizeof(saddr));
    listen(ssock, 1);
    csize = sizeof(caddr);
    csock = accept(ssock, (struct
sockaddr*)&caddr, (socklen_t*)&csize);
    recv(csock, msg, sizeof(msg), 0);
    send(csock, msg, strlen(msg), 0);
    close(csock); close(ssock);
    return 0;
}
```

Client

```
#include<stdio.h>
#include<string.h>
#include<sys/socket.h>
#include<arpa/inet.h>
#include<unistd.h>
int main(){
    int sock; struct sockaddr_in saddr;
    char msg[2000], cmsg[2000];
    sock = socket(AF_INET, SOCK_STREAM, 0);
    saddr.sin_family = AF_INET;
    saddr.sin_port = htons(2000);
    saddr.sin_addr.s_addr =
inet_addr("127.0.0.1");
    connect(sock, (struct sockaddr*)&saddr,
sizeof(saddr));
    printf("Msg: "); fgets(cmsg, sizeof(cmsg),
stdin);
    send(sock, cmsg, strlen(cmsg), 0);
    recv(sock, msg, sizeof(msg), 0);
    printf("Server: %s\n", msg);
    close(sock); return 0;
}
```

2. UDP Socket

Server

```
#include<stdio.h>
#include<string.h>
#include<sys/socket.h>
#include<arpa/inet.h>
int main(){
    int sock; struct sockaddr_in saddr, caddr;
    char msg[2000]; socklen_t clen =
sizeof(caddr);
    sock = socket(AF_INET, SOCK_DGRAM,
IPPROTO_UDP);
    saddr.sin_family = AF_INET;
    saddr.sin_port = htons(2000);
    saddr.sin_addr.s_addr =
inet_addr("127.0.0.1");
    bind(sock, (struct sockaddr*)&saddr,
sizeof(saddr));
    recvfrom(sock, msg, sizeof(msg), 0, (struct
sockaddr*)&caddr, &clen);
    sendto(sock, msg, strlen(msg), 0, (struct
sockaddr*)&caddr, clen);
    return 0;
}
```

Client

```
#include<stdio.h>
#include<string.h>
#include<sys/socket.h>
#include<arpa/inet.h>
int main(){
    int sock; struct sockaddr_in saddr;
    char msg[2000]; socklen_t slen =
sizeof(saddr);
    sock = socket(AF_INET, SOCK_DGRAM,
IPPROTO_UDP);
    saddr.sin_family = AF_INET;
    saddr.sin_port = htons(2000);
    saddr.sin_addr.s_addr =
inet_addr("127.0.0.1");
    fgets(msg, sizeof(msg), stdin);
    sendto(sock, msg, strlen(msg), 0, (struct
sockaddr*)&saddr, slen);
    recvfrom(sock, msg, sizeof(msg), 0, (struct
sockaddr*)&saddr, &slen);
    return 0;
}
```

3. Stop and Wait

Server

```
#include<stdio.h>
#include<string.h>
#include<sys/socket.h>
#include<arpa/inet.h>
int main(){
    int sock, csock, csize;
    struct sockaddr_in saddr, caddr;
    char msg[200];
    sock = socket(AF_INET, SOCK_STREAM,0);
    saddr.sin_family = AF_INET;
    saddr.sin_port = htons(6500);
    saddr.sin_addr.s_addr =
inet_addr("127.0.0.1");
    bind(sock, (struct sockaddr*)&saddr,
sizeof(saddr));
    listen(sock,1);
    csize = sizeof(caddr);
    csock = accept(sock, (struct sockaddr*)&caddr,
&csize);
    while(1){
        if(recv(csock, msg, 200, 0) <= 0) break;
        printf("Rcvd: %s\nAck(Y/N): ", msg);
        scanf("%s", msg);
        send(csock, msg, strlen(msg), 0);
        return 0;
    }
}
```

Client

```
#include<stdio.h>
#include<string.h>
#include<sys/socket.h>
#include<arpa/inet.h>
int main(){
    int sock, f, timer; struct sockaddr_in saddr;
    char msg[200], resp[200];
    sock = socket(AF_INET, SOCK_STREAM, 0);
    saddr.sin_family = AF_INET;
    saddr.sin_port = htons(6500);
    saddr.sin_addr.s_addr =
inet_addr("127.0.0.1");
    connect(sock, (struct sockaddr*)&saddr,
sizeof(saddr));
    printf("Frames: "); scanf("%d", &f);
    while(f > 0) {
        printf("Msg: "); scanf("%s", msg);
        timer = 5;
        while(timer > 0) {
            send(sock, msg, strlen(msg), 0);
            recv(sock, resp, 200, 0);
            if(resp[0] == 'Y' || resp[0] == 'y') {
                f--; break;
            } else { timer--; }
        }
        return 0;
    }
}
```

4. Go Back N

Server

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <netinet/in.h>
void sendAck(int s, int a){
    char b[80]; sprintf(b, 80, "%d", a);
    send(s, b, 80, 0);
}
int main(){
    int s, c, len, f, next=0; char b[80];
    struct sockaddr_in saddr, caddr;
    s = socket(AF_INET, SOCK_STREAM, 0);
    saddr.sin_family=AF_INET;
    saddr.sin_port=htons(8080);
    saddr.sin_addr.s_addr=INADDR_ANY;
    bind(s, (struct sockaddr*)&saddr,
sizeof(saddr));
    listen(s,5);
    len=sizeof(caddr); c=accept(s,(struct
sockaddr*)&caddr,&len);
    while(1){
        recv(c, b, 80, 0);
        if(!strcmp(b,"EXIT")) break;
        f = atoi(b);
        if(f != next){ sendAck(c, next-1);
continue; }
        if(rand()%3==0) continue; // Drop
        sendAck(c, f); next++;
    }
    return 0;
}
```

Client

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <sys/time.h>
int main(){
    int s, nf, ws, ack=-1, w1=0, w2, i, n; char
b[80];
    struct sockaddr_in saddr; struct timeval
tv={3,0};
    s = socket(AF_INET, SOCK_STREAM, 0);
    setsockopt(s, SOL_SOCKET, SO_RCVTIMEO, &tv,
sizeof(tv));
    saddr.sin_family=AF_INET;
    saddr.sin_port=htons(8080);
    connect(s, (struct sockaddr*)&saddr,
sizeof(saddr));
    scanf("%d %d", &nf, &ws); w2=ws-1;
    while(1){
        for(i=w1; i<=w2 && i<nf; i++){
            sprintf(b, 80, "%d", i); send(s, b,
80, 0);
        }
        while(1){
            n=recv(s, b, 80, 0);
            if(n>0){
                ack=atoi(b);
                if(ack==nf-1){ send(s,"EXIT",
80,0); return 0;}
                w1=ack+1; w2=w1+ws-1; break;
            } else break; // Timeout triggers
outer loop
        }
    }
}
```

5. Selective Repeat

Server

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <netinet/in.h>
void sendAck(int s, int a){
    char b[80]; sprintf(b, 80, "%d", a); send(s,
b, 80, 0);
}
int main(){
    int s, c, len, f, r[100]={0}, exp=0; char
b[80];
    struct sockaddr_in saddr, caddr;
    s = socket(AF_INET, SOCK_STREAM, 0);
    saddr.sin_family=AF_INET;
    saddr.sin_port=htons(8080);
    saddr.sin_addr.s_addr=INADDR_ANY;
    bind(s, (struct sockaddr*)&saddr,
sizeof(saddr));
    listen(s,5);
    len=sizeof(caddr); c=accept(s,(struct
sockaddr*)&caddr,&len);
    while(1){
        recv(c, b, 80, 0); if(!strcmp(b,"EXIT"))
break;
        f = atoi(b);
        if(rand()%3==0) continue; // Drop
        if(!r[f]) r[f]=1;
        sendAck(c, f);
        while(r[exp]) exp++;
    } return 0;
}
```

Client

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <netinet/in.h>
int main(){
    int s, nf, ws, w1=0, w2, i, n, a[100]={0};
    char b[80];
    struct sockaddr_in saddr; struct timeval
    tv={3,0};
    s = socket(AF_INET, SOCK_STREAM, 0);
    setsockopt(s, SOL_SOCKET, SO_RCVTIMEO, &tv,
    sizeof(tv));
    saddr.sin_family=AF_INET;
    saddr.sin_port=htons(8080);
    connect(s, (struct sockaddr*)&saddr,
    sizeof(saddr));
    scanf("%d %d", &nf, &ws); w2=w1+ws-1;
    while(w1<nf){
        for(i=w1; i<=w2 && i<nf; i++){
            if(!a[i]){ sprintf(b,80,"%d",i);
            send(s,b,80,0); }
        }
        while(1){
            n=recv(s, b, 80, 0);
            if(n>0){
                int ack=atoi(b); if(ack>=0 &&
                ack<nf) a[ack]=1;
                while(w1<nf && a[w1]) w1++;
                w2=w1+ws-1;
                if(w1>=nf){ send(s,"EXIT",80,0);
                return 0;}
                break;
            } else {
                for(i=w1; i<=w2 && i<nf; i++){ //
                Timeout resend
                if(!a[i]){ sprintf(b,
                80,"%d",i); send(s,b,80,0); }
                }
            }
        }
    } return 0;
}
```

6. FTP

Server

```
#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <fcntl.h>
#include <unistd.h>
int main(){
    int s, c, f, n; struct sockaddr_in sa, ca;
    char fname[100], fdata[300]; socklen_t
    csz=sizeof(ca);
    s=socket(AF_INET, SOCK_STREAM, 0);
    sa.sin_family=AF_INET;
    sa.sin_port=htons(2000);
    sa.sin_addr.s_addr=INADDR_ANY;
    bind(s, (struct sockaddr*)&sa, sizeof(sa));
    listen(s, 5);
    while(1){
        c=accept(s, (struct sockaddr*)&ca, &csz);
        n=read(c, fname, 100); fname[n]=0;
        f=open(fname, O_RDONLY);
        if(f<0) write(c, "Not found", 9);
        else { n=read(f,fdata,300);
        write(c,fdata,n); close(f); }
        close(c);
    } return 0;
}
```

Client

```
#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
int main(){
    int c, n; struct sockaddr_in sa; char fn[50],
    fd[300];
    c=socket(AF_INET, SOCK_STREAM, 0);
    sa.sin_family=AF_INET;
    sa.sin_port=htons(2000);
    sa.sin_addr.s_addr=inet_addr("127.0.0.1");
    connect(c, (struct sockaddr*)&sa, sizeof(sa));
    scanf("%s", fn); write(c, fn, 50);
    n=read(c, fd, 300); fd[n]=0;
    printf("Data: %s\n", fd); close(c); return 0;
}
```

7. DVR Algorithm

```
#include <stdio.h>
struct node { unsigned dist[20], from[20]; }
rt[10];
int main() {
    int cost[20][20], n, i, j, k, c;
    scanf("%d", &n);
    for(i=0; i<n; i++)
        for(j=0; j<n; j++) {
            scanf("%d", &cost[i][j]);
            if(i==j) cost[i][j]=0;
            rt[i].dist[j]=cost[i][j];
        }
    for(k=0; k<n; k++)
        for(i=0; i<n; i++) for(j=0; j<n; j++)
            if(rt[i].dist[j] > cost[i][k] +
            rt[k].dist[j]) {
                rt[i].dist[j] = rt[i].dist[j] +
                cost[i][k];
                rt[i].from[j] = k; c++;
            }
    while(c!=0);
    for(i=0; i<n; i++) {
        printf("\nR%d: ", i+1);
        for(j=0; j<n; j++)
            printf("N%d(via%d):%d ", j+1,
            rt[i].from[j]+1, rt[i].dist[j]);
        } return 0;
}
```

8. SMTP Client

```
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <arpa/inet.h>
void snd(int s, char *m) {
    send(s,m,strlen(m),0); char r[2000]; recv(s,r,
    2000,0);
}
int main(){
    int s=socket(AF_INET,SOCK_STREAM,0);
    struct sockaddr_in
    sa={.sin_family=AF_INET, .sin_port=htons(25)};
    sa.sin_addr.s_addr=inet_addr("127.0.0.1");
    connect(s,(struct sockaddr*)&sa,sizeof(sa));
    char r[2000]; recv(s,r,2000,0);
    snd(s, "HELO local\r\n");
    snd(s, "MAIL FROM:<a@b.com>\r\n");
    snd(s, "RCPT TO:<c@d.com>\r\n");
    snd(s, "DATA\r\n");
    snd(s, "Sub: Test\r\n\r\nMsg\r\n.\r\n");
    snd(s, "QUIT\r\n");
    return 0;
}
```

9. Leaky Bucket

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main() {
    int p[10], i, clk, b_sz, orate, rem=0, t, op;
    for(i=0; i<10; ++i) p[i]=rand()%100;
    scanf("%d %d", &orate, &b_sz);
    for(i=0; i<10; ++i) {
        if((p[i]+rem) > b_sz) printf("\nDrop %d",
        p[i]);
        else {
            rem+=p[i]; t=rand()%10;
            for(clk=10; clk<=t; clk+=10) {
                if(rem) {
                    op=(rem<=orate)?rem:orate;
                    rem-=op; printf("\nTx %d, Rem
                    %d", op, rem);
                }
            }
        } return 0;
    }
}
```